AZURE DATA FACTORY · GUIDE PAS À PAS · NIVEAU AVANCÉ

Construire un pipeline de production sur Azure Data Factory

Un pipeline maître qui découvre seul ses fichiers, un pipeline enfant appelé en parallèle, un Mapping Data Flow qui sépare les bonnes et les mauvaises lignes, et un déclencheur qui réagit au dépôt d'un fichier. Sans clé, sans mot de passe, sans planification.

Get Metadata · Filter · If Condition · Fail · ForEach · Execute Pipeline · Copy data · Pipeline return value · Append variable · Data flow · Set variable · Storage event trigger · Identité managée

Par **Abdoul Hamid Diallo** · Microsoft Certified Fabric Data Engineer, Fabric Analytics Engineer et Azure AI Engineer

3 → 1
trois fichiers CSV, un indicateur quotidien par station

84 + 6
sessions valides en gold, lignes défectueuses en quarantaine

0 secret
la fabrique s'authentifie avec sa propre identité

0 clic
le dépôt d'un fichier témoin lance tout le traitement

Le scénario

Un opérateur de bornes de recharge pour véhicules électriques exploite trois stations : **Metz Gare**, **Nancy Stanislas** et **Thionville Centre**. Chaque station dépose chaque jour le fichier CSV de ses sessions de recharge : borne, heure de début et de fin, énergie délivrée en kWh, prix payé, moyen de paiement.

Certaines lignes arrivent abîmées : une énergie négative, une énergie vide. La direction veut, chaque jour et par station, le nombre de sessions, l'énergie totale, le chiffre d'affaires, la durée moyenne et le prix moyen du kWh. Les lignes abîmées ne doivent ni fausser les chiffres ni disparaître : on doit pouvoir les montrer à la station concernée.

Le parcours

1. Préparer Azure : groupe, lac de données, fabrique, droits
2. Une connexion et deux datasets réutilisables
3. Le pipeline enfant : ingérer un fichier
4. Le data flow : typer, contrôler, agréger
5. Le pipeline maître : découvrir, protéger, orchestrer
6. Publier, exécuter et vérifier les chiffres
7. Le déclencheur événementiel
8. Tout fermer pour ne plus rien consommer

Ce qu'il vous faut

1. Un abonnement Azure. Le compte gratuit suffit.
2. Le rôle **Propriétaire** (Owner) sur cet abonnement, pour pouvoir attribuer un rôle à la fabrique.
3. Les quatre fichiers **joints à ce PDF** : `charging_metz.csv`, `charging_nancy.csv`, `charging_thionville.csv` et `_ready.txt`. Pour les extraire, ouvrez le panneau **Pièces jointes** (le trombone) de votre lecteur PDF, par exemple Adobe Acrobat Reader, puis enregistrez-les.

Comment lire ce guide

Chaque valeur à saisir figure dans un **bloc sombre**, à copier telle quelle. Les menus sont donnés en français, avec le libellé anglais entre parenthèses quand l'interface peut s'afficher dans les deux langues.

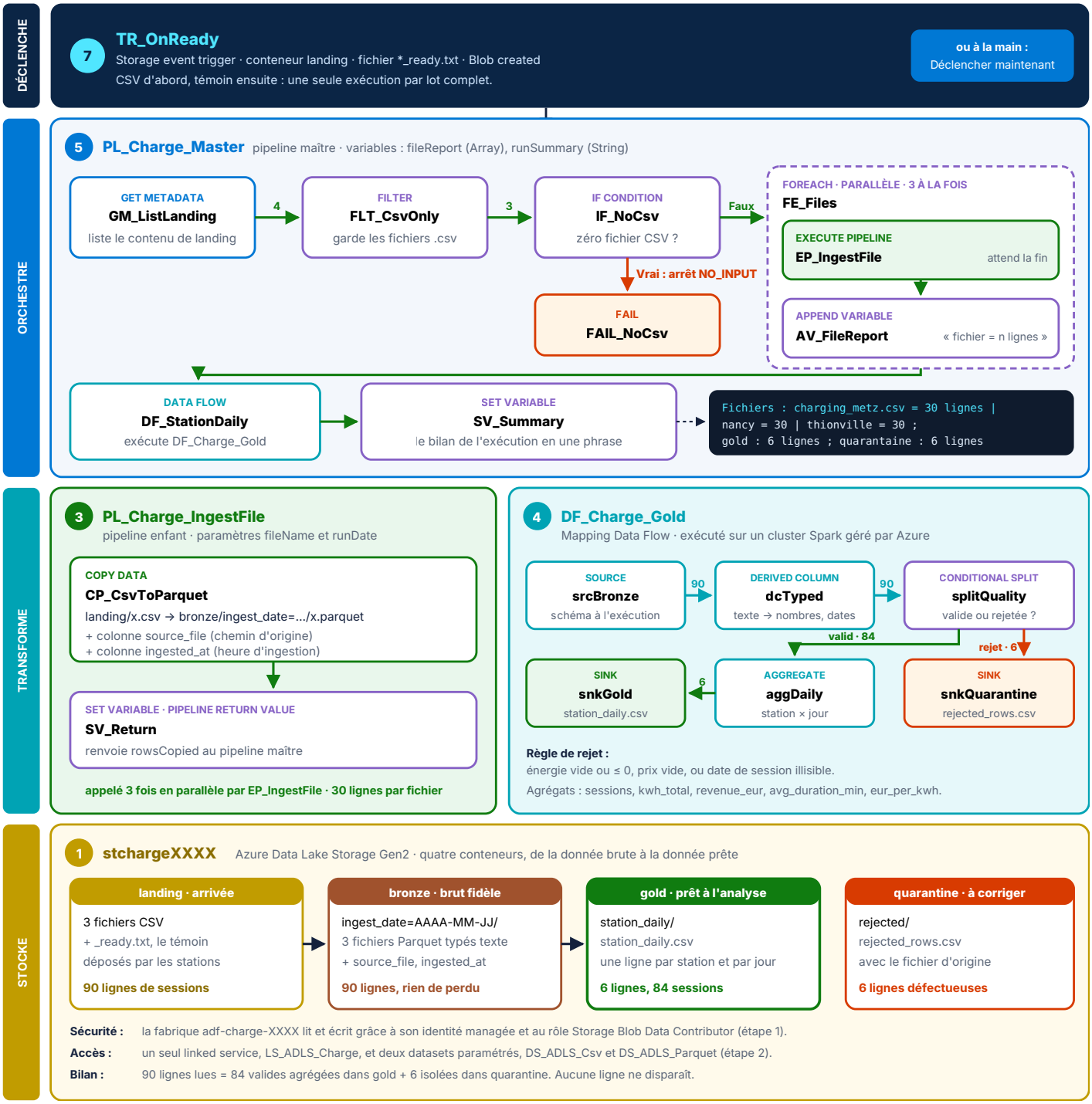
Remplacez `XXXX` par quatre chiffres de votre choix, toujours les mêmes : les noms du compte de stockage et de la fabrique doivent être uniques dans le monde entier.

Quand une case réclame une expression qui commence par `@`, cliquez dans la case puis sur **Ajouter du contenu dynamique** (Add dynamic content), collez l'expression et validez avec **OK**. Si une expression longue occupe deux lignes, collez-la sur une seule.

Ce qui distingue ce pipeline. Deux datasets génériques au lieu d'un par fichier. Une garde qui arrête proprement l'exécution si rien n'est arrivé. Un pipeline enfant qui renvoie une valeur à son parent. Une couche bronze partitionnée par date et traçable ligne par ligne. Une quarantaine plutôt qu'une suppression silencieuse. Un bilan lisible en une phrase. Ce sont les réflexes d'un pipeline qui part en production.

La carte complète du pipeline

Le traitement se lit de haut en bas, en quatre couches : **ce qui déclenche**, **ce qui orchestre**, **ce qui transforme** et **où vivent les données**. Les pastilles numérotées renvoient aux étapes du guide. Les nombres sur les flèches sont ceux que vous retrouverez à l'étape 6.



bleu : lecture, données · violet : contrôle du flux · vert : déplacement, succès · turquoise : transformation Spark · orange : erreur, rejet · or : stockage

Flèche verte pleine
« En cas de réussite » : l'activité suivante ne démarre que si la précédente a réussi. C'est le chemin normal.

Flèche orange
Le chemin d'erreur : une branche Vrai qui arrête l'exécution, ou les lignes rejetées par le contrôle qualité.

Cadre en pointillés
Un conteneur d'activités : tout ce qu'il contient se répète, ici une fois par fichier, trois à la fois.

Nombres sur les flèches
Le volume qui passe : 4 éléments listés, 3 CSV gardés, 90 lignes lues, 84 valides, 6 rejetées.

1 Préparer Azure : groupe, lac de données, fabrique, droits

Tout se passe dans le portail `portal.azure.com`. Choisissez la même région pour chaque ressource, par exemple **France Central** : les données ne quittent pas la région et les transferts restent gratuits.

1.1 Le groupe de ressources, votre bouton « tout effacer »

Dans la barre de recherche du portail, tapez le terme ci-dessous, ouvrez le service, puis cliquez sur **Créer** (Create).

```
Groupes de ressources
```

Groupe de ressources	`rg-adf-charge`

Région **France Central**, puis **Vérifier + créer** › **Créer**. Toutes les ressources du guide iront dans ce groupe : le supprimer à la fin supprimera tout d'un seul geste (étape 8).

1.2 Le compte de stockage, votre lac de données

Recherchez le service ci-dessous, puis **Créer**.

```
Comptes de stockage
```

Onglet **Informations de base** (Basics) : groupe de ressources **rg-adf-charge**, puis le nom (minuscules et chiffres uniquement) :

Nom du compte de stockage	`stchargeXXXX`

Région **France Central** · Service principal **Azure Blob Storage ou Azure Data Lake Storage Gen2** · Performances **Standard** · Redondance **Stockage localement redondant (LRS)**.

Onglet **Avancé** (Advanced) : cochez **Activer l'espace de noms hiérarchique** (Enable hierarchical namespace). Puis **Vérifier + créer** › **Créer** › **Accéder à la ressource**.

> **Pourquoi cette case ?** Sans elle, vous obtenez un simple stockage d'objets où les « dossiers » ne sont que des préfixes dans un nom. Avec elle, le compte devient un vrai lac de données Gen2 : de vrais dossiers, des droits par dossier et des renommages instantanés. C'est ce que Spark, Synapse, Fabric et Databricks attendent.

1.3 Les quatre conteneurs et les fichiers source

Dans le compte : **Stockage des données** › **Conteneurs** › **+ Conteneur**. Créez-les un par un, dans cet ordre :

```
landing
```

```
bronze
```

```
gold
```

```
quarantine
```

Ouvrez **landing** › **Charger** (Upload) et déposez les trois fichiers CSV. **Gardez `_ready.txt` de côté** : il servira de déclencheur à l'étape 7.

1.4 La fabrique de données

Recherchez le service ci-dessous, puis **Créer**.

```
Fabriques de données
```

Onglet **Informations de base** : groupe **rg-adf-charge**, région **France Central**, version **V2**, et le nom :

Nom de la fabrique	`adf-charge-XXXX`

Onglet **Configuration Git** : cochez **Configurer Git plus tard**. Puis **Vérifier + créer** › **Créer**. Azure donne aussitôt à la fabrique une **identité managée** : un compte Microsoft Entra ID qui porte son nom et qu'aucun humain ne connaît.

1.5 Autoriser la fabrique à lire et écrire dans le lac

Retournez dans **stchargeXXXX** › **Contrôle d'accès (IAM)** › **+ Ajouter** › **Ajouter une attribution de rôle**. Cherchez puis sélectionnez le rôle :

```
Storage Blob Data Contributor
```

Suivant. Attribuer l'accès à : **Identité managée** › **+ Sélectionner des membres** › Identité managée : **Data factory (V2)** › choisissez **adf-charge-XXXX** › **Sélectionner** › **Vérifier + attribuer**, deux fois.

> **Pourquoi une identité managée ?** Aucune clé de compte, aucune chaîne de connexion, aucun secret à stocker ni à renouveler. La fabrique prouve qui elle est auprès d'Entra ID, et le rôle dit ce qu'elle a le droit de faire. C'est la configuration exigée par les équipes sécurité en production.

> **Attention.** Une attribution de rôle met un peu de temps à se propager. Si le test de connexion de l'étape 2 échoue juste après, patientez un instant puis relancez-le.

1.6 Activer les événements de stockage

Recherchez **Abonnements** › votre abonnement › **Paramètres** › **Fournisseurs de ressources**. Filtrez sur le nom ci-dessous. S'il n'est pas à l'état **Inscrit** (Registered), sélectionnez-le et cliquez sur **Inscrire**.

```
Microsoft.EventGrid
```

Le déclencheur de l'étape 7 repose sur Azure Event Grid ; l'inscrire maintenant évite de l'attendre plus tard. Ouvrez enfin **adf-charge-XXXX** › **Lancer le studio** (Launch studio). Le reste du guide se déroule dans ADF Studio : **Auteur** (Author, le crayon) pour construire, **Surveillance** (Monitor, la jauge) pour suivre, **Gérer** (Manage, la boîte à outils) pour les connexions et les déclencheurs.

2 Une connexion et deux datasets réutilisables

2.1 Le linked service : la seule porte vers le lac

Gérer › **Services liés** › **+ Nouveau**. Cherchez **Azure Data Lake Storage Gen2** › **Continuer**.

Nom
```
LS_ADLS_Charge
```

Type d'authentification **System-assigned managed identity** · Méthode de sélection du compte **À partir de l'abonnement Azure** › votre abonnement › **stchargeXXXX**. Cliquez sur **Tester la connexion** : le résultat doit être vert. Puis **Créer**.

2.2 Le dataset CSV paramétré

Auteur › **+** › **Jeu de données** › **Azure Data Lake Storage Gen2** › **Continuer** › **DelimitedText** › **Continuer**.

Nom
```
DS_ADLS_Csv
```

Service lié **LS_ADLS_Charge** · cochez **La première ligne comme en-tête** · Importer le schéma **Aucun** · laissez le chemin vide › **OK**. Onglet **Paramètres** › **+ Nouveau**, trois fois, type **String**, valeur par défaut vide :

```
container
```
```
folder
```
```
file
```

Onglet **Connexion** › **Chemin du fichier** : trois cases. Dans chacune, **Ajouter du contenu dynamique** puis collez :

case 1, le conteneur
```
@dataset().container
```
case 2, le dossier
```
@dataset().folder
```
case 3, le fichier
```
@dataset().file
```

Vérifiez : délimiteur **Virgule (,)**, en-tête coché, **Valeur null** (Null value) vide. Une cellule vide deviendra ainsi un vrai *null*, utile au contrôle qualité de l'étape 4.

2.3 Le dataset Parquet paramétré

Même création, format **Parquet**, service lié **LS_ADLS_Charge**, schéma **Aucun**. Mêmes trois paramètres, même chemin dynamique, compression **snappy**.

Nom
```
DS_ADLS_Parquet
```

Pourquoi deux datasets seulement ? Un dataset par format plutôt qu'un par fichier : chaque activité choisit elle-même conteneur, dossier et fichier. Ces deux objets servent six fois ici, et serviraient pour cent fichiers.

Règle d'or. Dans une activité, un paramètre de dataset ne reste jamais vide, sinon la validation échoue. Pour une chaîne vide, saisissez `@string('')` en contenu dynamique.

3 Le pipeline enfant : ingérer un fichier, et le dire

Il reçoit un nom de fichier, le convertit en Parquet dans bronze et renvoie le nombre de lignes copiées. **Auteur** › **+** › **Pipeline**, puis clic sur la toile vide › onglet **Paramètres** › **+ Nouveau**, deux fois, type **String**.

nom du pipeline
```
PL_Charge_IngestFile
```
paramètre 1
```
fileName
```
paramètre 2
```
runDate
```

CP_CsvToParquet

Copy data

Glissez **Copier les données** sur la toile et nommez-la : elle lit un CSV dans landing et l'écrit en Parquet dans bronze.

Source : jeu de données **DS_ADLS_Csv**, type de chemin **Chemin d'accès du fichier dans le jeu de données** (File path in dataset), puis ses trois paramètres :

container
```
landing
```
folder (contenu dynamique)
```
@string('')
```
file (contenu dynamique)
```
@pipeline().parameters.fileName
```

Source › **Colonnes supplémentaires** (Additional columns) › **+ Nouveau**, deux fois. Colonne 1 : valeur **$$FILEPATH** choisie dans la liste. Colonne 2 : **Ajouter du contenu dynamique**.

colonne 1 : nom
```
source_file
```
colonne 2 : nom
```
ingested_at
```
colonne 2 : valeur
```
@utcNow()
```

Récepteur (Sink) : jeu de données **DS_ADLS_Parquet**, puis ses trois paramètres. **Mappage** : n'importez rien, la copie reprend les colonnes du fichier.

container
```
bronze
```
folder (contenu dynamique)
```
@concat('ingest_date=',
pipeline().parameters.runDate)
```
file (contenu dynamique)
```
@replace(pipeline().parameters.fileName,
'.csv', '.parquet')
```

Pourquoi ? Le dossier `ingest_date=AAAA-MM-JJ` suit la convention Hive : Spark, Synapse et Fabric y lisent la date comme une colonne. `source_file` et `ingested_at` rendent chaque ligne traçable.

SV_Return

Set variable · Pipeline return value

Glissez **Définir une variable**, reliez **CP_CsvToParquet** à elle (flèche verte **En cas de réussite**). Onglet **Paramètres** : **Valeur de retour du pipeline** (Pipeline return value) › **+ Nouveau**, type **Expression**.

Clé
```
rowsCopied
```
Valeur
```
@activity('CP_CsvToParquet').output.rowsCopied
```

Cliquez sur **Valider** : aucune erreur attendue. Tout reste en brouillon jusqu'à **Tout publier** (étape 6).

4 Le data flow : typer, contrôler, agréger

Un Mapping Data Flow se dessine comme un schéma et s'exécute sur un cluster Spark qu'Azure démarre et arrête pour vous. **Auteur** › **+** › **Flux de données**. N'activez **pas** le **Débogage du flux de données** : le pipeline l'exécutera directement. Chaque transformation s'ajoute avec le petit **+** en bas à droite de la précédente.

Nom du data flow
```
DF_Charge_Gold
```

srcBronze — Source

Ajouter une source. Type de source **Jeu de données** · jeu de données **DS_ADLS_Parquet** · cochez **Autoriser la dérive de schéma** (Allow schema drift). N'importez aucune projection.

Nom du flux de sortie
```
srcBronze
```

> **Pourquoi la dérive de schéma ?** Le data flow ne connaît pas les colonnes à l'avance : il les découvre à l'exécution. Si demain une station ajoute une colonne, rien ne casse.

dcTyped — Derived column

Dans bronze, tout est encore du texte. Cette étape crée cinq colonnes typées avec `byName()`, qui lit une colonne par son nom même si le schéma n'a pas été importé. Ajoutez chaque colonne avec **+ Ajouter** › **Ajouter une colonne** : saisissez le nom, puis l'expression.

Nom du flux de sortie
```
dcTyped
```

station_name
```
toString(byName('station'))
```

session_date
```
toDate(substring(toString(byName('start_time')), 1, 10))
```

energy
```
toDouble(byName('energy_kwh'))
```

price
```
toDouble(byName('price_eur'))
```

duration_min
```
(toLong(toTimestamp(toString(byName('end_time')), 'yyyy-MM-dd HH:mm:ss')) -
toLong(toTimestamp(toString(byName('start_time')), 'yyyy-MM-dd HH:mm:ss'))) /
60000.0
```

La durée se calcule en millisecondes entre la fin et le début de la session, puis se convertit en minutes en divisant par 60 000.

splitQuality — Conditional split

Le contrôle qualité. **Fractionner sur** : **Première condition correspondante** (First matching condition). Le premier flux reçoit les lignes rejetées ; le flux par défaut (**Lignes qui ne remplissent aucune condition**, Rows that do not meet any condition) reçoit les lignes valides.

Nom du flux de sortie
```
splitQuality
```

Flux 1 : nom
```
rejected
```

Flux 1 : condition
```
isNull(energy) || energy <= 0 || isNull(price) ||
isNull(session_date)
```

Flux par défaut : nom
```
valid
```

aggDaily — Aggregate

Ajoutez-le derrière le flux **valid**. Onglet **Regrouper par** (Group by) : **station_name**, puis **+** et **session_date**. Onglet **Agrégats** (Aggregates) : cinq colonnes, le nom puis l'expression.

Nom du flux de sortie
```
aggDaily
```

sessions
```
count()
```

kwh_total
```
round(sum(energy), 2)
```

revenue_eur
```
round(sum(price), 2)
```

avg_duration_min
```
round(avg(duration_min), 1)
```

eur_per_kwh
```
round(sum(price) / sum(energy), 3)
```

Le prix moyen du kWh se calcule comme un total divisé par un total, et non comme une moyenne de prix unitaires : c'est la seule formule juste quand les sessions n'ont pas la même taille.

snkGold — Sink

Derrière **aggDaily**. Jeu de données **DS_ADLS_Csv**. Onglet **Paramètres** : option de nom de fichier **Sortie dans un seul fichier** (Output to single file), cochez **Effacer le dossier** (Clear the folder). Onglet **Optimiser** (Optimize) : **Partition unique** (Single partition).

Nom
```
snkGold
```

Fichier
```
station_daily.csv
```

snkQuarantine — Sink

Derrière le flux **rejected** de splitQuality. Mêmes réglages : **DS_ADLS_Csv**, sortie dans un seul fichier, effacer le dossier, partition unique.

Nom
```
snkQuarantine
```

Fichier
```
rejected_rows.csv
```

À respecter. Les noms `snkGold` et `snkQuarantine` sont relus par le pipeline maître pour compter les lignes écrites : gardez exactement la même casse. La **Partition unique** est obligatoire pour obtenir un seul fichier.

5 Le pipeline maître : découvrir, protéger, orchestrer

Auteur › **+** › **Pipeline**. Cliquez sur la toile vide › onglet **Variables** › **+ Nouveau**, deux fois : `fileReport` de type **Array**, puis `runSummary` de type **String**. Reliez chaque activité à la suivante par la flèche verte **En cas de réussite**.

nom du pipeline
```
PL_Charge_Master
```

variable Array
```
fileReport
```

variable String
```
runSummary
```

GM_ListLanding — Get Metadata

Dresse la liste de tout ce que contient landing.

Jeu de données **DS_ADLS_Csv**. **Liste des champs** › **+ Nouveau** › **Éléments enfants** (Child items).

container
```
landing
```

folder
```
@string('')
```

file
```
@string('')
```

FLT_CsvOnly — Filter

Ne garde que les fichiers CSV : le témoin _ready.txt et les éventuels dossiers sont écartés.

Éléments
```
@activity('GM_ListLanding').output.childItems
```

Condition
```
@and(equals(item().type, 'File'),
endswith(toLower(item().name), '.csv'))
```

IF_NoCsv — If Condition

La garde : si aucun CSV n'est arrivé, on s'arrête net et on le dit.

Expression
```
@equals(activity('FLT_CsvOnly').output.FilteredItemsCount,
0)
```

Onglet **Activités** › **Vrai** › crayon › glissez une activité **Échec** (Fail). Laissez **Faux** vide, puis revenez au pipeline par le fil d'Ariane, en haut.

FAIL_NoCsv — Fail, dans la branche Vrai

Un échec volontaire, avec un message et un code lisibles par une alerte ou un collègue.

Message
```
Aucun fichier CSV dans le conteneur landing :
exécution arrêtée.
```

Code d'erreur
```
NO_INPUT
```

FE_Files — ForEach

Relié à IF_NoCsv par la flèche verte. Traite les fichiers trois par trois, en parallèle. Onglet **Paramètres** : décochez **Séquentiel** (Sequential).

Nombre de lots (Batch count)
```
3
```

Éléments
```
@activity('FLT_CsvOnly').output.Value
```

Onglet **Activités** › crayon. À l'intérieur de la boucle, ajoutez les deux activités suivantes, puis revenez au pipeline par le fil d'Ariane.

EP_IngestFile — Execute Pipeline

Pipeline appelé **PL_Charge_IngestFile** · cochez **Attendre la fin** (Wait on completion), sans quoi la valeur de retour n'existe pas encore.

fileName
```
@item().name
```

runDate
```
@formatDateTime(pipeline().TriggerTime,
'yyyy-MM-dd')
```

AV_FileReport — Append variable

Après EP_IngestFile. Variable **fileReport**. Ajoute « fichier = n lignes » au rapport.

Valeur
```
@concat(item().name, ' = ',
string(activity('EP_IngestFile').output.pipelineReturnValue.rowsCopied),
' lignes')
```

DF_StationDaily — Data flow

Relié à FE_Files. Onglet **Paramètres** : flux de données **DF_Charge_Gold** · taille de calcul **Petit** (Small). Les paramètres de ses trois datasets apparaissent dans le même onglet.

Flux	container	folder	file
srcBronze	`bronze`	`@concat('ingest_date=', formatDateTime(pipeline().TriggerTime, 'yyyy-MM-dd'))`	`@string('')`
snkGold	`gold`	`station_daily`	`@string('')`
snkQuarantine	`quarantine`	`rejected`	`@string('')`

SV_Summary — Set variable

Relié à DF_StationDaily. Variable **runSummary**. Assemble le bilan de toute l'exécution en une phrase.

```
@concat('Fichiers : ', join(variables('fileReport'), ' | '), ' ; gold : ',
string(activity('DF_StationDaily').output.runStatus.metrics.snkGold.rowsWritten), ' lignes ; quarantaine : ',
string(activity('DF_StationDaily').output.runStatus.metrics.snkQuarantine.rowsWritten), ' lignes')
```

Pourquoi un pipeline enfant ? ADF interdit de placer un ForEach dans un If. Le découpage maître et enfant contourne cette limite, rend l'ingestion réutilisable par d'autres pipelines et isole chaque fichier : un échec sur Thionville apparaît comme une exécution distincte, sans masquer Metz ni Nancy.

6 Publier, exécuter et vérifier les chiffres

1. En haut : **Tout valider** (Validate all), puis **Tout publier** (Publish all) › **Publier**. Avant la publication, rien n'existe côté service.
2. Ouvrez **PL_Charge_Master** › **Ajouter un déclencheur** › **Déclencher maintenant** (Trigger now) › **OK**.
3. **Surveillance** › **Exécutions de pipeline**. Le maître et trois exécutions de l'enfant apparaissent ; la colonne **État** doit afficher **Opération réussie** sur chaque ligne. Le data flow démarre d'abord son cluster Spark : c'est normal qu'il prenne plus de temps que le reste.
4. Cliquez sur le nom **PL_Charge_Master**. Dans la liste des activités, survolez **SV_Summary** et cliquez sur l'icône **Sortie** (Output). Le champ `value` doit contenir ce bilan (l'ordre des fichiers peut varier) :

```
Fichiers : charging_metz.csv = 30 lignes | charging_nancy.csv = 30 lignes | charging_thionville.csv = 30 lignes ; gold : 6 lignes ; quarantaine : 6 lignes
```

Dans le portail : **stchargeXXXX** › **Navigateur de stockage** (Storage browser) › **gold** › **station_daily** › **station_daily.csv** › **Modifier** pour l'aperçu :

station_name	session_date	sessions	kwh_total	revenue_eur	avg_duration_min	eur_per_kwh
Metz Gare	2026-10-12	14	429.51	177.51	58.9	0.413
Metz Gare	2026-10-13	14	377.19	155.62	62.0	0.413
Nancy Stanislas	2026-10-12	14	417.32	183.26	62.9	0.439
Nancy Stanislas	2026-10-13	14	369.61	162.74	55.7	0.44
Thionville Centre	2026-10-12	14	415.98	155.74	47.4	0.374
Thionville Centre	2026-10-13	14	467.30	174.71	51.4	0.374

Contrôle global : 84 sessions, 2 476.91 kWh, 1 009.58 €. L'ordre des lignes peut changer. La quarantaine contient les six lignes `*-9001` (énergie négative) et `*-9002` (énergie vide), chacune avec son fichier d'origine. Dans bronze, le dossier `ingest_date=` du jour contient trois fichiers Parquet.

7 Le déclencheur événementiel

Plutôt que de planifier un traitement « tous les jours à 6 h » en espérant que les fichiers soient là, on réagit à leur arrivée. **Gérer** › **Déclencheurs** › **+ Nouveau** :

Nom	Conteneur	Le chemin se termine par (Blob path ends with)
`TR_OnReady`	`landing`	`_ready.txt`

Type **Événements de stockage** (Storage events) · compte **stchargeXXXX** · événement **Objet blob créé** (Blob created) · Ignorer les objets blob vides **Non** · cochez **Démarrer le déclencheur à la création** (Start trigger on creation) › **Continuer** › **Continuer** › **OK**. L'aperçu des fichiers correspondants peut être vide : c'est normal. Ouvrez ensuite **PL_Charge_Master** › **Ajouter un déclencheur** › **Nouveau/Modifier** › **TR_OnReady** › **OK**, puis **Tout publier**. Déposez `_ready.txt` dans landing : une exécution lancée par **TR_OnReady** apparaît dans **Surveillance**, avec le même bilan.

> **Le motif du fichier témoin.** Les stations déposent d'abord leurs CSV, puis un petit fichier qui signifie « lot complet ». Déclencher sur le témoin, et non sur chaque CSV, évite trois exécutions concurrentes sur un lot à moitié arrivé.

Si quelque chose bloque

Symptôme	Remède
« Aucune valeur fournie pour le paramètre »	Un paramètre est resté vide : saisissez `@string('')`.
Test de connexion en erreur 403	Rôle pas encore propagé, ou attribué ailleurs que sur le compte de stockage.
Déclencheur refusé à la création	`Microsoft.EventGrid` non inscrit (1.6).
FAIL_NoCsv se déclenche	Les CSV ne sont pas à la racine de landing.
Erreur sur pipelineReturnValue	**Attendre la fin** décoché, ou clé différente de `rowsCopied`.
Erreur sur metrics.snkGold	Nom du sink différent, casse comprise.
Un seul fichier refusé	Onglet **Optimiser** : **Partition unique**.
Colonne introuvable dans le data flow	Dérive de schéma décochée, ou faute dans un `byName`.

8 Tout fermer

Une ressource oubliée continue de coûter. Selon que vous gardez le projet ou non :

Pour le garder en sommeil

1. **Gérer** › **Déclencheurs** › survolez **TR_OnReady** › **Arrêter** (■), puis **Tout publier**. L'état doit afficher **Arrêté**.
2. Vérifiez que le **Débogage du flux de données** est éteint : un cluster de débogage allumé est facturé tant qu'il tourne.

Pour tout effacer

1. Portail › **Groupes de ressources** › **rg-adf-charge** › **Supprimer le groupe de ressources**.
2. Recopiez le nom pour confirmer, puis **Supprimer**. La fabrique, le lac, les données et l'abonnement Event Grid disparaissent ensemble.

```
rg-adf-charge
```

Pour contrôler : **Gestion des coûts** › **Analyse des coûts**, filtré sur rg-adf-charge. Les montants y apparaissent avec un léger décalage.